Reproducibility guide

Repository source: REPRODUCIBILITY.md

Contents

This guide explains how to reproduce the symbolic checks, numerical cross-checks, manuscript build, and typeset Markdown documentation from a fresh clone.

Continuous-integration environments

The GitHub Actions workflow uses:

- Ubuntu 24.04;
- Python 3.13;
- the exact Python package versions in `requirements.txt`;
- R 4.6.1, using base R only;
- Pandoc 3.10.1;
- XeLaTeX, `latexmk`, and the TeX packages installed by the workflow; and
- `pdfinfo`, `pdffonts`, and `pdftotext` from Poppler for PDF preflight.

The workflow file is `.github/workflows/verification.yml`.

Fresh-clone procedure

Clone the repository and enter its root directory:

```
git clone YOUR_REPOSITORY_URL
cd minvol-degree3-spectral-bound
```

Replace `YOUR_REPOSITORY_URL` with the repository's actual clone URL.

1. Python checks

Create and activate a virtual environment if desired, then install the pinned requirements and run the suite:

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
./run_python_checks.sh
```

A successful run ends with all of the following:

```
EXACT DEGREE-3 IDENTITY CHECK: PASS
ALTERNATIVE COMPACT-FORM CHECK: PASS
PAIRING-COUNT CHECK: PASS
BOUND ARITHMETIC CHECK: PASS
NUMERICAL STRESS TESTS: PASS
```

2. R checks

The R scripts use base R only:

```
./run_r_checks.sh
```

A successful run contains:

```
R PAIRING-COUNT CHECK: PASS
R NUMERICAL STRESS TESTS: PASS
```

The runner records both outputs and the R session information in `expected/`.

3. Manuscript build

With Python 3, `latexmk`, and a standard LaTeX installation available:

```
./build_paper.sh
```

The resulting manuscript is:

```
paper/degree3_spectral_refinement.pdf
```

4. Markdown-derived PDFs

With Pandoc 3 or later, Python 3, and XeLaTeX available:

```
./build_markdown_pdfs.sh
```

The rendered files are written beneath `rendered/markdown/`, preserving the source directory structure. Their SHA-256 hashes are written to:

```
rendered/markdown/SHA256SUMS.txt
```

On macOS, verify the tracked files with:

```
shasum -a 256 -c rendered/markdown/SHA256SUMS.txt
```

On systems with GNU coreutils, use:

```
sha256sum -c rendered/markdown/SHA256SUMS.txt
```

5. Metadata and PDF preflight

Check consistency of the author name, AI-system identification, citation file, and licensing declaration:

```
./ci/preflight_metadata.sh
```

With Poppler installed, validate the manuscript and every Markdown-derived PDF:

```
./ci/preflight_pdfs.sh
```

The PDF check confirms that each PDF is nonempty and readable, has at least one page, contains extractable text, uses embedded fonts, and has a valid checksum when applicable. It also confirms that the manuscript contains the full author name, candidate coefficient, AI-assistance disclosure, and model designation.

What is exact and what is numerical

The general degree-3 tensor identities are checked with exact symbolic arithmetic by the Python/SymPy programs. The R programs are independently written numerical and finite-enumeration cross-checks. Agreement between the two languages helps detect implementation and convention errors, but it is not a substitute for mathematical review of the proof.

PDF reproducibility

The build scripts fix the source-date epoch and normalize PDF trailer identifiers. Repeated builds with an unchanged toolchain are intended to be byte-for-byte reproducible. Different operating systems, TeX Live releases, fonts, or Pandoc versions may still produce visually equivalent but byte-different PDFs. The editable Markdown and LaTeX sources remain the source of truth.

GitHub Actions

The workflow runs automatically on pushes to `main`, pull requests, and manual requests from the Actions tab. It stores the Python and R logs and the generated PDFs as temporary workflow artifacts for inspection. A green run means that the repository's checks and builds completed in the declared environments; it is not an independent proof review.

Reporting a discrepancy

Record the following when reporting a failed or differing run:

- commit hash;
- operating system and architecture;
- Python, SymPy, NumPy, R, Pandoc, and TeX versions;
- the full command used; and
- the complete console output beginning at the first warning or error.